PickTiming

실시간 가격 비교 · 구매 타이밍 추천 플랫폼

RPA로 가격을 모으고, ML로 구매 시점을 점수화하고,
LLM 응답을 검증해 쉬운 말로 설명합니다.

박미금 · 조장·PM·데이터/AI

김모란 · 백엔드·배포

이오리 · 프런트엔드·UI/UX

홍기흥 · RPA·데이터 수집

기간 2026.10.21 – 11.05 (저장소 커밋 기준) · 팀 4명

github.com/fsclass-n/picktiming

picktiming.onrender.com (운영) · picktiming.netlify.app (데모)





실제 UI · 상품 상세 / 관리자 AI 사용량 (데모 데이터)

프로젝트 한눈에 보기

4,133
애플리케이션 코드(줄)
백엔드 2,112 · 프런트 1,532 · RPA 489

34
REST API
/api/v1 · 6개 라우터

12
DB 테이블
TiDB · MySQL 호환

71
자동 검사
백엔드 51 · RPA 11 · 브라우저 9

문제

- 판매처마다 다른 가격·배송비
- 매일 바뀌는 가격
- 현재 최저가만으로는 구매 시점 판단이 어려움

해결

- 1시간 주기 RPA 가격 수집
- 실구매가 비교 + 90일 가격 흐름
- 적기 점수(0~100) + LLM 설명
- 목표가·판정 변화 알림

결과

- Render 운영 배포·헬스 체크
- Netlify 데모(같은 API 계약)
- CI: SQLite·MySQL 8·Docker
- 회원·관리자 권한 분리

팀 구성과 역할 분담

박 박미금
조장 · PM · 데이터/AI

- 일정·위험 관리, 발표
- 판정 피처 8종·모델·점수
- Gemini 프롬프트·응답 검증

주요 코드
backend/app/ml · services/timing.py · ai/gemini.py

김 김모란
백엔드 · 배포

- FastAPI API·인증·권한
- 수집 수신 API·알림 생성
- Render·EC2·CI/CD

주요 코드
backend/app/routers · security.py · .github/

이 이오리
프런트엔드 · UI/UX

- React 통합 UI·차트·모달
- 3단계 가입·관리자 화면
- 접근성·반응형 브라우저 검사

주요 코드
frontend/src/App.jsx · pages/Admin* · tests/

홍 홍기흥
RPA · 데이터 수집

- 수집기·파서·네이버 API
- 정제·이상치 제외
- 스케줄 실행·주간 리포트

주요 코드
rpa/collector · jobs/weekly_report.py

협업: 통합 연계·테스트·오류 수정은 전원 참여 · main push = 운영 배포이므로 CI 통과 후 반영 · 비밀값은 환경 변수·GitHub Secrets

문제 정의와 요구사항, 그리고 제약

사용자 문제

- 같은 상품, 판매처별 가격·배송비 상이
- 가격은 매일 변동
- '지금 사도 될까?' 판단 근거 부족
- 가격 확인을 반복해야 함

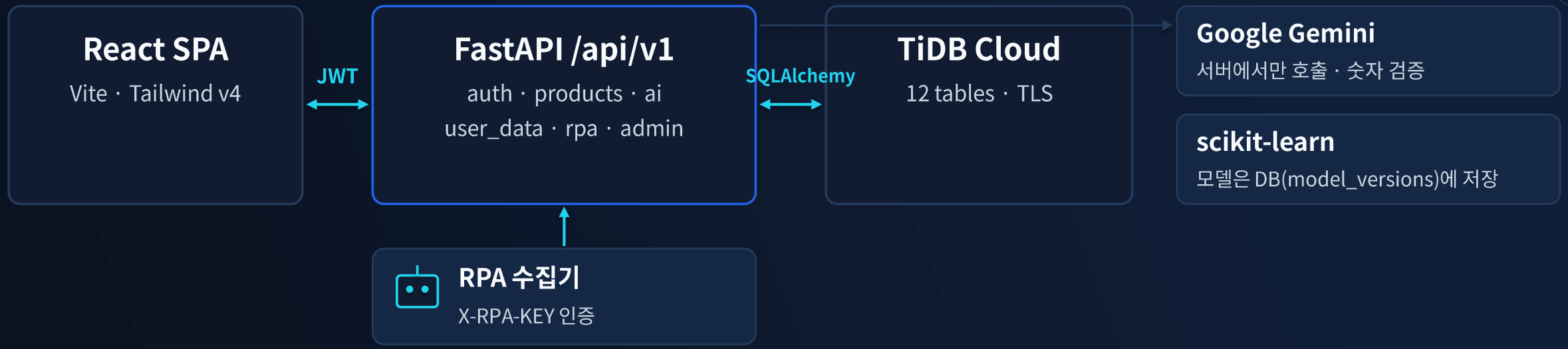
핵심 요구사항

- 기능 17건: 비교·차트·판정·AI·알림·관리
- 비기능 8건: 성능 목표·보안·비용·접근성
- 이력 30일 미만은 '판단 보류'
- LLM 답변은 입력 수치만 사용
- 관리자 권한은 서버에서 검사

제약 조건

- 약관·robots.txt 준수, 요청 간격 3초
- LLM 비용·환각 위험
- Render 무료 플랜: 상주 작업 불가·슬립
- 로컬·Render·EC2가 운영 DB 공유

시스템 아키텍처와 배포 구성



배포 토폴로지 (서로 호출하지 않는 독립 구성)

Netlify
React 빌드만 · 브라우저 목업 API
데모

Render
루트 Dockerfile · React + FastAPI
운영 확인

AWS EC2
compose: nginx + api + rpa
구성 완료·운영 미확인

GitHub Actions
CI · 변경 경로별 배포 · 매시 07분 수집
CI 통과

기술 선택 이유와 트레이드오프

영역	선택	이유	트레이드오프 · 보완
API	FastAPI	Pydantic 입력 검증, 자동 /docs, 의존성 주입으로 권한 분리	대규모 관리자 기능은 직접 구현
DB	TiDB + MySQL 8(CI) + SQLite	MySQL 호환 무료 서버리스, 테스트는 빠른 SQLite + 호환성 MySQL	공유 운영 DB → 초기화 금지 규칙
화면	React · Vite · Tailwind v4	필터·모달·가입 단계 등 상태 관리, 관리자 화면 지연 로드	Vanilla JS 대비 빌드 단계 추가
ML	RandomForest + TimeSeriesSplit	표 형식 피처·적은 데이터에 적합, 시간 순서 분할로 미래 정보 누수 방지	데이터 부족 시 규칙 확률로 대체
LLM	Gemini 구조화 출력	JSON 스키마로 파싱 안정, temperature 0.3, 숫자 대조 검증	실패 시 템플릿 → 표현 다양성 감소
배포	Render + EC2 + Actions	무료 플랜은 상주 작업 불가 → 예약 수집 분리, 환경 독립	콜드 스타트·중복 수집 → 시간대 중복 방지

수집에서 알림까지, 한 번의 요청으로

① **대상 조회**
GET /rpa/listings

→

② **수집**
robots.txt · 3초 간격
재시도 3·6·12초

→

③ **정제**
중복 제거
±50% 이상치 제외

→

④ **적재**
POST /rpa/prices/bulk
같은 시간대 중복 skip

→

⑤ **판정**
timing_predictions
사전 계산

→

⑥ **알림**
목표가·판정 변화
24시간 중복 방지

스케줄 (EC2: 컨테이너 내장 스케줄러 / Render: GitHub Actions 예약)

매시
가격 수집 collect
EC2 정각 · Actions 07분

매일 03:00
모델 재학습 train
F1이 같거나 좋을 때만 교체

매일 06:00
참가격 public
주소·키 없으면 건너뜀

월 07:00
주간 리포트 report
엑셀 + 차트

핵심: 가격 저장과 판정·알림을 같은 트랜잭션에서 처리해, 화면은 미리 계산된 점수를 바로 읽습니다(판정 캐시 2시간).

구매 시점을 점수로: 적기 판정 모델

적기 점수 = (1 − 7일 내 5% 하락 확률) × 70
+ (1 − 90일 가격 백분위) × 30

피처 8종

90일 백분위	최저가 대비	7일 평균 괴리	30일 평균 괴리
30일 기울기	30일 변동성	하락 후 경과일	월말 여부

- 라벨: 7일 안에 현재가 대비 5% 이상 하락하면 1
- RandomForest(150, depth 8, balanced) · TimeSeriesSplit(5)
- 검증 F1이 운영 모델 이상일 때만 교체(100행 미만이면 생략)
- 활성 모델이 없으면 규칙 확률, 이력 30일 미만은 판단 보류

실제 코드 발췌

```
# backend/app/services/timing.py
model, mv = load_active(db)
if model is not None:
    prob = float(model.predict_proba(
        pd.DataFrame([row[FEATURES]]))[0, 1])
else:
    prob, version = heuristic_prob(row), "heuristic"
pct = float(row["percentile_90d"])
score = int(round((1 - prob) * 70 + (1 - pct) * 30))
label = label_for(score)  # BUY_NOW / WAIT_A_BIT / WAIT
```

한계와 다음 단계

70/30 가중치는 기획 단계 정의값이며 실데이터 검증이 필요합니다.
다음 단계: 판정 후 7일 실제 가격으로 적중률 집계·공개.

LLM 답변을 믿을 수 있게: 숫자 검증과 대체 경로

분석 사실
점수·가격·추세

→

Gemini
JSON 스키마
temperature 0.3

→

숫자 대조
입력에 없는 숫자?

→

1회 재요청
위반 숫자를 알려 줌

→

대체 응답
정해진 설명

실제 코드 발췌

```
# backend/app/ai/gemini.py  (_generate 요약)
out, tok = _call(client, system, schema, payload)
bad = unknown_numbers(to_text(out), facts)
if bad:
    out, tok2 = _call(..., extra=f"... {', '.join(bad)} ...")
    meta["validation"] = "RETRY"
    if unknown_numbers(to_text(out), facts):
        out, meta["validation"] = None, "FALLBACK"
# except: 429 / network / schema -> "ERROR"
if result is None:
    result = fallback()        # template_advice / template_chat
```

운영 장치

- 검증 상태 기록: OK · RETRY · FALLBACK · ERROR
- 조언 캐시: (상품·가격·판정·모델) SHA-256, 24시간
- 사용자별 일일 한도(기본 20회), 초과 시 429
- 키는 서버에만, 브라우저 직접 호출 금지
- 가짜 클라이언트로 재요청·대체·오류 경로 테스트

API 설계와 데이터 무결성

인증 경계 분리

- 사용자: Authorization Bearer JWT
- 수집기: X-RPA-KEY, hmac.compare_digest 비교
- 관리자: ADMIN 역할 의존성
- 운영에서 기본 비밀값이면 서버 기동 거부

관리자 권한 보호 규칙

- 본인 권한 변경 금지 (SELF_ROLE_CHANGE)
- 마지막 관리자 강등 금지 (LAST_ADMIN)
- 테스트 관리자 조회 전용 (DEMO_READ_ONLY)
- 오류 응답 {code, message} 형식 통일

같은 시간대 중복 저장 방지 (멱등 적재)

```
# backend/app/routers/rpa.py  POST /rpa/prices/bulk
hour = at.replace(minute=0, second=0, microsecond=0)
seen = set(db.execute(select(PriceHistory.listing_id).where(
    PriceHistory.listing_id.in_(ids),
    PriceHistory.collected_at >= hour,
    PriceHistory.collected_at < hour + timedelta(hours=1))).scalars())
for i in body.items:
    if i.listing_id in seen:
        duplicate += 1; continue
    seen.add(i.listing_id)
    db.add(PriceHistory(..., total_price=i.price + i.shipping_fee))
db.flush()
alerts = sum(refresh_product(db, p) for p in touched)
db.commit()
```

두 수집기(EC2·Actions)가 같은 DB에 보내도 행·알림이 중복되지 않습니다.
검증: test_bulk_prices_skip_same_hour

CI/CD: 바뀐 폴더만 배포한다

CI (push · PR)

backend	backend-mysql	rpa	frontend	docker
Ruff · pytest (SQLite)	pytest on MySQL 8	Ruff · pytest	npm ci · React 빌드	이미지 빌드 · /health 확인

변경 경로 → 배포 대상 (detect-changes.sh)

frontend/	→ Netlify · Render · EC2(web)
backend/ · requirements.txt	→ Render · EC2(api)
rpa/	→ EC2(rpa)
Dockerfile · render.yaml	→ Render
docs/ · .claude/	→ 배포 없음

배포 방식

- Render: Auto-Deploy ‘After CI Checks Pass’
- EC2: SSH → git pull → 바뀐 서비스만 compose 재빌드
- Netlify: Git 연동, React 빌드 결과만 게시
- Render 수집: Actions cron ‘7 * * * *’, /health로 깨운 뒤 실행
- MySQL 8 job이 SQLite에서 안 보이는 차이를 사전 발견

하나의 화면 코드로 데모와 운영을 모두



실제 UI · 7·30·90일 가격 흐름과 구매 타이밍 신호등 (데모 데이터)

```
// frontend/src/lib/api.js
const BASE = (window.APP_CONFIG?.API_BASE_URL || '').trim();
// BASE == ''  -> handleMock(): same REST contract in browser

// frontend/src/App.jsx
const Admin = lazy(() => import('./pages/Admin.jsx'));
const AdminAI = lazy(() => import('./pages/AdminAI.jsx'));
```

설계 포인트

- 데모 모드: 같은 API 계약을 브라우저 목업으로 처리
- Render: FastAPI가 SPA 제공, /js/config.js는 같은 도메인
- 고객 차트는 가벼운 SVG, Plotly는 관리자에서만 지연 로드

접근성·반응형

- 모달 Tab 순환·Escape·포커스 복귀
- 본문 바로가기, 화면 이동 시 본문 포커스
- 375·768·1440px 가로 넘침 없음 검사

예의 있는 수집기: 허용된 만큼, 천천히, 다시

실제 코드 발췌

```python
# rpa/collector/http.py
def fetch(self, url, render=False, wait_for=None):
    if not self.allowed(url):          # robots.txt
        raise CollectError("robots.txt disallow: " + url)
    for attempt in range(self.retries):        # 3
        self.wait()                            # 3s interval
        try:
            if render:                         # Playwright
                return render_page(url, self.ua, ...)
            r = self.s.get(url, timeout=self.timeout)
            r.raise_for_status()
            return BeautifulSoup(r.text, "html.parser")
        except Exception:
            time.sleep(3 * 2 ** attempt)      # 3, 6, 12s
    raise CollectError("request failed: " + url)
```

robots.txt를 확인할 수 없으면 수집하지 않음(Disallow로 간주)

수집·정제

- 등록 판매처(malls.json) · 네이버 쇼핑 API(키 설정 시)
- JavaScript 페이지는 Playwright로 렌더링
- 직전 가격 대비 ±50% 초과는 이상치 제외
- 시연 쇼핑몰은 실제 접속 없이 모의 가격

두 실행 경로

- EC2: rpa 컨테이너 내장 스케줄러(정각)
- Render: GitHub Actions 예약(07분)
- 같은 코드 · 같은 API · 시간대 중복 방지

문제 해결 사례

무료 플랜에서 스케줄러를 돌릴 수 없음

원인 Render Free는 상주 작업 불가·15분 무요청 시 슬립

해결 GitHub Actions 매시 예약 + /health로 깨운 뒤 수집

결과 Render도 독립적으로 가격 갱신

두 수집기가 같은 DB에 저장

원인 EC2·Actions가 같은 시간대에 같은 가격 전송 → 중복 행·알림

해결 판매처·시간대 기준 두 번째 가격 skip

결과 test_bulk_prices_skip_same_hour 통과

LLM이 입력에 없는 숫자를 말함

원인 모델이 일반 지식으로 수치를 보충

해결 숫자 대조 → 1회 재요청 → 템플릿 대체, 상태 기록

결과 재요청·대체·오류 테스트 3건 통과

SQLite에선 통과, MySQL에선 실패

원인 DB마다 행 선택 순서가 달라 가격 없는 판매 페이지가 선택됨

해결 CI MySQL 8 job으로 발견, 테스트 대상 선택 수정

결과 TiDB 호환성 문제를 배포 전 차단

테스트와 품질 관리

브라우저 E2E · 9개 시나리오

Playwright · 핵심 사용자 흐름

통합 (API) · 백엔드 51건

API · 인증 · 권한 · AI 검증 · 수집 API · SPA 제공 · DB TLS

단위 · RPA 11건 + 피처 · 검증기

파싱 · 정제 · 스케줄 · 네이버 · S3, 피처 · 숫자 검증

CI

- Ruff
- SQLite
- MySQL 8
- Docker

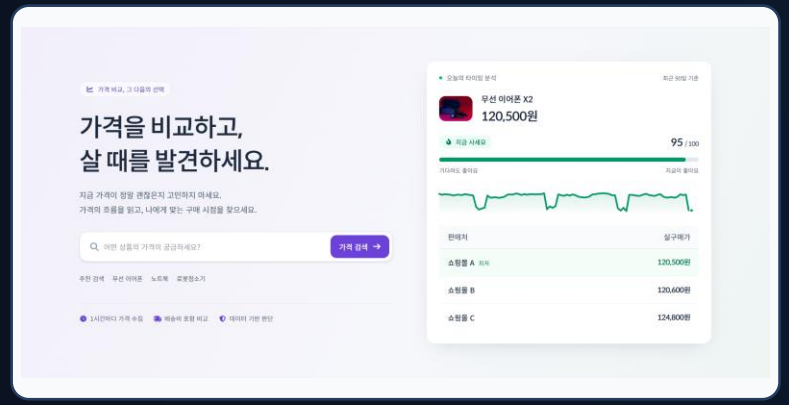
테스트 격리

- conftest가 DATABASE_URL을 임시 SQLite로 덮어씀
- 운영 TiDB 초기화 명령은 금지 규칙으로 관리

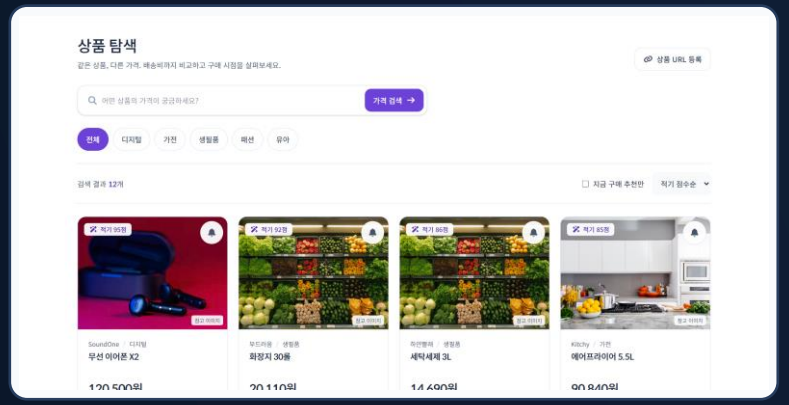
외부 의존성 대체

- Gemini: 가짜 클라이언트로 재요청 · 대체 검증
- OAuth: 토큰 교환을 가짜 응답으로 검증

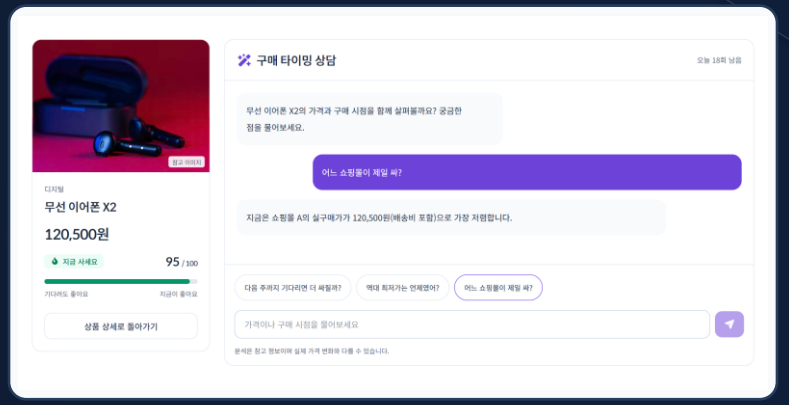
실제 동작하는 서비스



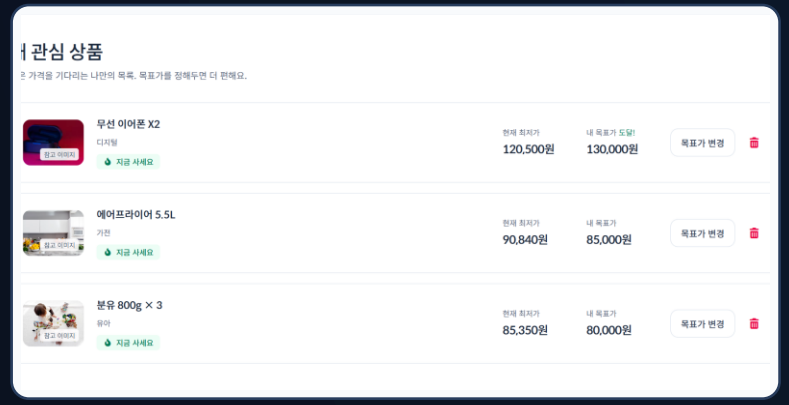
홈 · 가격 탐색



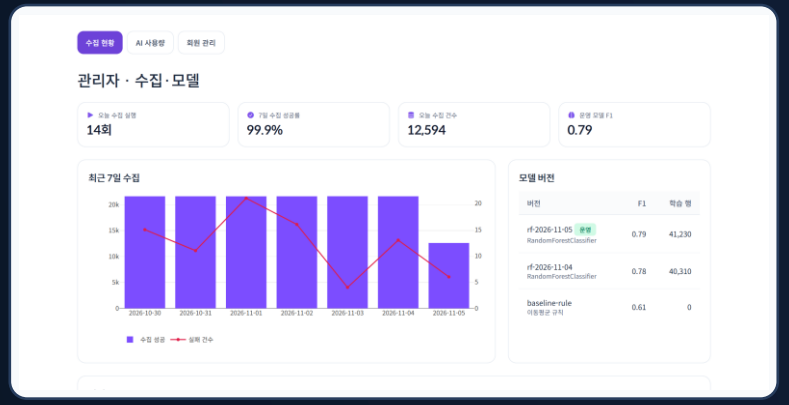
검색·필터



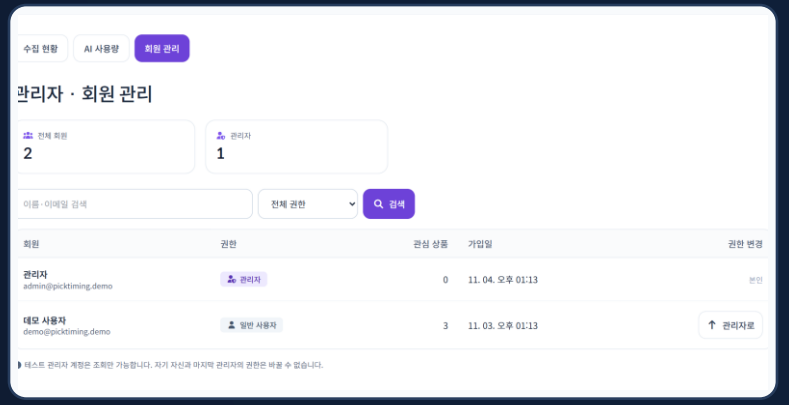
AI 상담



관심 상품·목표가



관리자 수집·모델



회원·권한 관리

운영 picktiming.onrender.com · 데모 picktiming.netlify.app · 화면 데이터는 데모

회고: 잘한 점, 한계, 다음 단계

잘한 점

- LLM 출력을 검증하는 구조
- 제약에서 출발한 배포 설계
- 운영 DB와 분리된 테스트
- 데모·운영이 같은 API 계약

한계 (솔직하게)

- 모델 F1·AI 품질 실데이터 미측정
- EC2 운영 연결 미확인
- 토큰을 localStorage에 저장
- 알림은 사이트 내 알림만
- 공유 DB 백업·복구 절차 미정

다음 단계

- 판정 적중률 집계·공개
- 응답 시간·부하 실측
- HttpOnly 쿠키 인증 검토
- 모바일 푸시(PWA)
- 카드·쿠폰 실결제가

가격 비교를 넘어,
구매 타이밍까지.



코드와 문서로 확인할 수 있습니다

GitHub	github.com/fsclass-n/picktiming
운영	picktiming.onrender.com
데모	picktiming.netlify.app
문서	docs/md · docs/산출물

Q & A